AniAtlas — Probationary Period 1 Report

A platform for the discovery and cataloguing of independent web animation

Team	Nurlan Bekov (SE-2406), Madiyar Armish (SE-2420)
Supervisor	Akbota Rakym
Programme	Software Engineering, Astana IT University
Repository	https://github.com/nuulun/AniAtlas
Version	tag `prob-period-1` — https://github.com/nuulun/AniAtlas/tree/prob-period-1
YouTrack	https://aniatlas.youtrack.cloud/issues/ANI
Prototype	React source: https://github.com/nuulun/AniAtlas/tree/prob-period-1/prototype (React + Vite; run with `npm install && npm run dev`)
Diagram sources	https://github.com/nuulun/AniAtlas/tree/prob-period-1/diagrams/src
Date	24 October 2026

Table of contents

1. Project Proposal
   1.1 Problem statement
   1.2 Target users
   1.3 Aim
   1.4 Objectives
   1.5 Expected outcomes
   1.6 Scope
   1.7 Engineering complexity
   References
2. Evidence of Relevance and User Interviews
   2.1 Analysis of available data
   2.2 User interviews
   2.3 Findings
   2.4 Assumptions and their status
   2.5 Requirements-change summary
   2.6 Limitations and next steps
3. Literature Review
   3.1 Scope and method
   3.2 Critical review
   3.3 Analysis table of publications
   3.4 Limitations the project takes into account
   References
4. Existing System Comparison
   4.1 Selected systems
   4.2 Comparison
   4.3 Evidence
   4.4 Conclusion
5. Technology Comparison
   5.1 Server platform
   5.2 Data storage
   5.3 Component interaction
   5.4 Frontend
   5.5 Recommendation method
   5.6 Other choices
6. Requirements Specification
   6.1 Actors
   6.2 Functional requirements
   6.3 Non-functional requirements
   6.4 Traceability
7. Architecture and Diagrams
   7.1 System context diagram
   7.2 Use case diagram
   7.3 Component diagram (draft)
   7.4 Data model
   7.5 Recommendation algorithm
   7.6 Alignment check
8. UX / Prototype
   8.1 Key user scenarios covered
   8.2 Component structure
   8.3 Interaction with the external system (YouTube)

8.4 Next steps
9. Verification and Evaluation Plan
9.1 Testing plan
9.2 Outcome metrics
9.3 Measurement conditions
9.4 Baselines for comparison
9.5 Evidence produced
9.6 Traceability: Must requirements → tests → evidence
10. Project Management and Risks
10.1 Tools and workflow
10.2 Allocation of responsibilities
10.3 Milestones
10.4 Dependencies
10.5 Risks
11. Supervisor Feedback and Feedback Log

1. Project Proposal

Project title	AniAtlas — a platform for the discovery and cataloguing of independent web animation
Team	Madiyar Armish (SE-2420) — backend, SRE/DevOps Nurlan Bekov (SE-2406) — frontend, co-architect
Supervisor	Akbota Rakym
Programme	Software Engineering, Astana IT University

1.1 Problem statement

Independent creators increasingly publish animated series, films and shorts directly on open platforms, above all YouTube, instead of going through studios and premium streaming services. Some of these works reach very large audiences:

- The pilot of *The Amazing Digital Circus* by the independent studio GLITCH passed 350 million views on YouTube [S2]. Videos related to the series on the GLITCH channel have more than 750 million views [S1].
- *Hazbin Hotel* started as a self-published YouTube pilot in 2019 and was released as a full series on Amazon Prime Video in January 2024 [S4].

Most independent projects, however, stay small and hard to find, for three reasons:

1. **No structured metadata.** On YouTube a series is just a playlist or a set of videos. Genre, format (series, film or short), release status and episode order are not available as searchable fields.
2. **Popularity-biased discovery.** YouTube's recommender is optimised for engagement on very large interaction data [2] and tends to amplify already-popular content [6]. Creators describe its behaviour as opaque and stressful [8].
3. **No specialised catalogue.** MyAnimeList, the largest animation database, only accepts professionally produced animation from Japan (and China and Korea), so independent Western web animation is excluded [S7]. General databases such as IMDb list some web series, but among millions of films and TV shows and without in-platform viewing.

Problem: there is no single specialised platform where viewers can find, compare, rate and track independent animated web series, and where the authors of these series can be discovered by an interested audience.

1.2 Target users

Group	Description	Main needs
Viewers	Teenagers and young adults who watch animation online and look for new series beyond mainstream streaming services	Find series by genre and tags, watch them, track progress, get recommendations
Independent authors and studios	Creators who publish animation on YouTube	Have their series listed with proper metadata and reach new viewers
Moderators	Platform staff	Keep the catalogue accurate: approve submissions, manage genres and tags

1.3 Aim

To design and develop a web platform that aggregates independent animated web content published on YouTube and lets users discover, compare, rate, track and watch it through the official embed player, with personalised recommendations based on genre preferences.

1.4 Objectives

Each objective has a measurable success criterion that is checked in the verification plan (Section 9).

#	Objective	Measurable success criterion
O1	Analyse user needs and existing systems, and specify requirements	3 user interviews (more planned for pre-defense 2); at least 4 systems compared on at least 10 criteria; every *Must* requirement has a testable acceptance criterion
O2	Design the data model, architecture and REST API	ER model, system context, use case and component diagrams, and an OpenAPI specification covering all *Must* requirements
O3	Implement the catalogue with full-text search and filters by genre, tags, format, status and year	Search p95 latency < 500 ms on 10,000 titles; precision@5 ≥ 0.8 on 20 test queries
O4	Integrate the YouTube Data API v3 to import series from playlists	At least 50 real independent titles and 300 episodes imported; daily API usage < 10,000 quota units
O5	Provide playback through the official YouTube embed player	100% of available episodes play on the title page; no video is stored on AniAtlas servers; unavailable videos are detected and marked
O6	Implement accounts, ratings, reviews, watch lists and comparison of 2–3 titles	All related acceptance tests (FR-9… FR-12) pass in CI
O7	Implement content-based recommendations with a cold-start strategy	Offline precision@10, nDCG@10, coverage and long-tail share measured against popularity and random baselines (hypothesis: content-based ranking scores higher); new users get recommendations immediately after choosing 3 genres
O8	Containerise, test and evaluate the system	Starts with one `docker compose up`; backend line coverage ≥ 60%; SUS usability score ≥ 70 with 5–8 test users

1.5 Expected outcomes

- A working MVP web platform (React frontend, Spring Boot backend, PostgreSQL) deployable with one `docker compose up`.
- A catalogue of at least 50 real independent animated titles imported via the YouTube Data API.
- A recommendation module whose quality has been measured against a popularity baseline.
- Documentation: requirements, architecture, test and evaluation results, and the diploma thesis.

1.6 Scope

In scope (MVP)	Out of scope
YouTube as the only content source	Other sources such as Vimeo or Bilibili (possible future work)
Web application (desktop and mobile browsers)	Native mobile applications
Catalogue, full-text search, filters, sorting	Hosting, downloading or re-uploading video
Playback through the official YouTube embed player	Monetisation, payments, advertising
Accounts, ratings, reviews, watch lists, comparison	Social features: friends, chat, comment threads
Submission of series by playlist URL with moderator approval	Automatic discovery of new series without a submission
Content-based recommendations by genres and tags	Collaborative or neural recommenders (future work)
Docker Compose deployment and CI	Large-scale production hosting and scaling

1.7 Engineering complexity

Area	Why it is non-trivial
Data model and search	Many-to-many relationships (titles–genres–tags–authors), a series → seasons → episodes hierarchy, and combinable filters with full-text search under a latency target
External API integration	The YouTube Data API has a default quota of 10,000 units per day [S5]. Import and synchronisation need batching (50 IDs per request), caching, scheduled jobs and quota accounting
Data consistency	Videos can be deleted, made private or made non-embeddable at any time. The system must detect this and update episode availability
Recommendations	Building user preference profiles, ranking by similarity, handling cold start [5], and evaluating offline against a baseline with accuracy and coverage metrics [7]
Security	Authentication with JWT and role-based access for viewers, authors and moderators
Architecture and operations	A modular backend, a single-page React client, containerised deployment and a CI pipeline with automated tests

References

Full reference list and analysis: Section 3 — Literature review.

- [2] Covington, P., Adams, J., Sargin, E. (2016). Deep neural networks for YouTube recommendations. *Proc. ACM RecSys*, 191–198. https://doi.org/10.1145/2959100.2959190
- [5] Panda, D. K., Ray, S. (2022). Approaches and algorithms to mitigate cold start problems in recommender systems: a systematic literature review. *Journal of Intelligent Information Systems*, 59(2), 341–366. https://doi.org/10.1007/s10844-022-00698-5
- [6] Klimashevskaia, A., Jannach, D., Elahi, M., Trattner, C. (2024). A survey on popularity bias in recommender systems. *User Modeling and User-Adapted Interaction*. https://arxiv.org/abs/2308.01118
- [7] Zangerle, E., Bauer, C. (2022). Evaluating recommender systems: Survey and framework. *ACM Computing Surveys*, 55(8). https://doi.org/10.1145/3556536
- [8] Choi, Y., Kang, E. J., Lee, M. K., Kim, J. (2023). Creator-friendly algorithms: Behaviors, challenges, and design opportunities in algorithmic platforms. *Proc. CHI '23*.

https://doi.org/10.1145/3544548.3581386
- [S1] YouTube Culture & Trends (2024). Set in motion: The new wave of indie animation. https://blog.youtube/culture-and-trends/indie-animation/
- [S2] YouTube Official Blog (2024). The Amazing Digital Circus. https://blog.youtube/creator-and-artist-stories/amazing-digital-circus/
- [S4] Amazon MGM Studios (2023). Prime Video announces premiere date and guest cast for Hazbin Hotel. https://press.amazonmgmstudios.com/us/en/press-release/prime-video-announces-premiere-date-and-guest-cast
- [S5] Google for Developers. YouTube Data API overview (quota). https://developers.google.com/youtube/v3/getting-started
- [S7] MyAnimeList forum: database guidelines discussion. https://myanimelist.net/forum/message/58190915?goto=topic

2. Evidence of Relevance and User Interviews

This section separates **verified findings** (from available data and from interviews) from **assumptions** that still need checking.

2.1 Analysis of available data

#	Finding	Source
D1	The pilot of *The Amazing Digital Circus* (GLITCH, an independent studio) passed 350 million views on YouTube	YouTube Official Blog, 2024 [S2] (company blog, not peer-reviewed)
D2	Videos related to the series on the GLITCH channel have more than 750 million views	YouTube Culture & Trends, 2024 [S1] (company blog)
D3	*Hazbin Hotel* started as a self-published YouTube pilot (2019) and became a series on Amazon Prime Video (2024)	Amazon MGM Studios press release [S4]
D4	MyAnimeList only lists professionally produced animation from Japan, China and Korea	MAL database guidelines, quoted on the MAL forum [S7]
D5	General recommender systems tend to amplify already-popular items	Peer-reviewed survey [6]; empirical YouTube study (preprint) [14]

2.2 User interviews

Method. Three semi-structured interviews with viewers, October 2026, 15–20 minutes each, conducted in Russian by the team. Respondents are anonymised as R1–R3; no names or contact details are stored. Notes are kept in `docs/interviews/`.

Question guide

1. How often do you watch animation, and where?
2. Do you watch independent animated series or shorts?
3. How did you find the last independent animated series you watched?
4. Do you look for such series? What makes finding and following them difficult?
5. Do you use IMDb, MyAnimeList, Letterboxd or similar services? What is missing there for web animation?
6. What do you consider when choosing what to watch?
7. Do you trust ratings and recommendations from other viewers?

Respondents and key statements

	Profile	Key statements
R1	Occasional viewer; prefers studio animation (Pixar), sometimes anime	Only independent title seen: *The Amazing Digital Circus*. Fan animations “find me” through YouTube recommendations or friends; “without algorithms or word of mouth I would hardly come across these works”. Does not use tracking services. Chooses by friends’ advice and popularity; would watch independent animation more often if it appeared in the information space.
R2	Frequent viewer who draws inspiration from animation; watches mostly on YouTube	Independent animation is a favourite because of new ideas and styles. Found the last series by chance in YouTube recommendations because of interest in a related game. Large creators are easy to follow, but “good animation gets lost because of YouTube’s recommendation algorithms”. Rarely uses IMDb; does not trust other viewers’ ratings; chooses by story and purpose of the work.
R3	Rare viewer; YouTube and sometimes streaming services	Watches independent animation rarely because it “rarely comes into view”. The last series was recommended by a friend. Independent series are scattered across platforms, have no advertising, and algorithms favour popular content. Irregular releases make it easy to miss new episodes. IMDb-like services lack new-episode information and many independent projects. Would like to follow a series, be notified about new episodes and see where to watch. Chooses by genre, theme, humour and visual style; trusts specific reviews from acquaintances; decides after the first episode.

2.3 Findings

Finding	Evidence	Effect on requirements
F1. Independent animation is found by chance, through the YouTube algorithm or friends	R1, R2, R3	Confirms the core problem; dedicated catalogue (FR-1…FR-6)
F2. Small creators are hidden by popularity-driven recommendations	R2, R3; D5	Content-based ranking with explanations (FR-16, FR-17); popularity used only as a baseline
F3. Following irregular releases is hard; new episodes are missed	R3	Watch lists with progress (FR-11); **FR-20 notifications raised from Could to Should**

Finding	Evidence	Effect on requirements
F4. Viewers choose by genre, theme, humour and visual style	R2, R3	Genre and tag filters; tags and artwork visible on cards (FR-5)
F5. Generic ratings are trusted little; friends' advice and specific text reviews matter more	R1, R2, R3	Text reviews (FR-10); "because you like …" explanations instead of a bare score
F6. Existing databases are not used for web animation	R1, R2, R3	Confirms the gap in Section 4

2.4 Assumptions and their status

#	Assumption	Status after interviews
A1	Viewers find it hard to discover independent animated series	**Confirmed** (3/3)
A2	Viewers want to filter by genre, tags and format	**Partly confirmed** (R3, R2 choose by genre/theme/story; R1 by popularity)
A3	Viewers are willing to watch through an embedded player	**Not verified**: to be tested in the usability study before pre-defense 2
A4	Viewers would use watch lists and episode tracking	**Partly confirmed** (R3 strongly; R1, R2 do not track today)
A5	Genre preferences are a sufficient basis for recommendations	**Open**: mixed trust in recommendations; to be checked by the offline evaluation

2.5 Requirements-change summary

Change	Reason
FR-20 (new-episode notifications) raised from *Could* to *Should*	F3
Recommendations must show an explanation, not just a score (FR-17 acceptance criterion)	F5
Usability test must include a task on embedded playback	A3 not verified

2.6 Limitations and next steps

Three respondents form a small convenience sample of viewers only. Before pre-defense 2 the team will add interviews with at least one independent creator or catalogue moderator and run task-based usability sessions with the prototype (see Section 11, items 1 and 7).

3. Literature Review

3.1 Scope and method

The review answers three questions:

- **Q1.** How is independent web content produced and discovered, and what problems do its creators face?
- **Q2.** Which recommendation approaches fit a small catalogue with sparse user data?
- **Q3.** How should search, recommendation and the platform as a whole be evaluated?

Sources were searched in Google Scholar, the ACM Digital Library, SpringerLink, SAGE Journals and arXiv. The search terms were *web series*, *YouTube creators*, *content-based recommendation*, *cold start*, *popularity bias*, *recommender evaluation* and *multimedia recommendation*. Most of the core sources are from 2021–2025. Foundational works such as Lops et al. 2011, Gomez-Uribe & Hunt 2015 and Covington et al. 2016 are kept because they define the methods used. Platform documentation and industry reports (S1–S7) are used only as evidence of facts, not as scientific sources.

The search was carried out in October 2026.

- **Inclusion criteria:** peer-reviewed journal and conference papers, academic monographs and handbooks in English; directly related to Q1–Q3; published 2015–2025, with older works allowed only if they define a method used in the project. One recent arXiv preprint [14] is included because it gives the newest empirical data on YouTube recommendations; it is marked as a preprint in the table.
- **Exclusion criteria:** papers without access to the full text; work on unrelated domains (e.g. news or e-commerce recommendation) without transferable findings.
- **Non-scientific sources:** blog posts and platform documentation are not counted as scientific sources and are used only as supporting evidence (S1–S7).

3.2 Critical review

Q1 — Independent web content and its discovery

Christian [11] shows that web television developed as an “open” alternative to the Hollywood network system. Independent creators gained distribution without gatekeepers, but in return they bear the full burden of marketing and visibility. Cunningham and Craig [12] describe “social media entertainment” as a separate industry in which creators depend on platform infrastructure they do not control.

Recent empirical work confirms the inequality of this ecosystem. Rieder et al. [9] analysed about 153,000 YouTube channels and 137 million videos. Most of the economic value concentrates at the top of an “aspirational curve”, and smaller creators rely on crowdfunding. Bishop [10] and Choi et al. [8] show that creators experience the recommendation algorithm as opaque. They form “folk theories” about it and suffer stress from unexplained changes in visibility.

Implication for AniAtlas: a specialised catalogue with explicit, structured metadata gives independent animation a discovery channel that does not depend only on YouTube’s general algorithm.

Q2 — Recommendation approaches

Covington et al. [2] describe YouTube’s two-stage deep-learning recommender. It relies on billions of interaction events, which a small new platform does not have. Gomez-Uribe and Hunt [3] show that Netflix combines several algorithms, including genre-based rows. Klimashevskaia et al. [6] review

popularity bias: collaborative systems trained on interaction data systematically over-recommend popular items. This is exactly the bias that hides small independent creators.

Content-based filtering [1, 13] recommends items similar to those a user liked, using item features such as genres and tags. It works with few users and treats new and old items equally, but it is limited to the features that are available and tends towards over-specialisation. Deldjoo et al. [4] review multimedia recommenders and show that richer content features (visual, audio, text) can improve on metadata-only approaches, at a much higher computational cost. Panda and Ray [5] review cold-start solutions. For new users, asking for initial preferences (for example, favourite genres at sign-up) and hybridising content-based and collaborative signals are the most common practical approaches.

Implication for AniAtlas: the MVP uses content-based filtering on genre and tag vectors. Cold start for new users is handled by preference elicitation at sign-up [5]. Collaborative and multimedia features are left for later stages, once enough data exists [4, 13].

Q3 — Evaluation

Zangerle and Bauer [7] propose a framework for recommender evaluation. It distinguishes offline metrics (accuracy such as precision@k and nDCG; beyond-accuracy metrics such as coverage and diversity) from user studies. It also stresses comparison against simple baselines. Klimashevskaia et al. [6] recommend measuring popularity bias explicitly, for example by the share of long-tail items in recommendations. Cakmak et al. [14] show empirically that YouTube Shorts recommendations disproportionately promote high-view videos. This supports using a popularity-based recommender as the baseline for comparison.

Implication for AniAtlas: the verification plan (Section 9) compares the content-based recommender with a popularity baseline using precision@10, catalogue coverage and long-tail share, and adds a small user study.

3.3 Analysis table of publications

#	Source	Year	Type	Approach / data	Key findings	Limitations	Use in AniAtlas
1	Lops, de Gemmis, Semeraro	2011	Book chapter	Review of content-based recommenders	Item features + user profile; good for new items	Over-specialisation; depends on feature quality	Basis of the recommender
2	Covington, Adams, Sargin	2016	Conference (RecSys)	Deep two-stage recommender at YouTube	Scales to billions of events	Needs huge interaction data	Why YouTube’s approach is not reproducible; motivation
3	Gomez-Uribe, Hunt	2015	Journal (ACM TMIS)	Netflix recommender as a business system	Several algorithms, genre rows, A/B tests	Industrial, few reproducible details	Genre-based rows on the home page
4	Deldjoo et al.	2020	Journal (ACM CSUR)	Survey of multimedia recommenders	Content features improve cold-item recommendations	High compute cost	Future work: thumbnail/description features
5	Panda, Ray	2022	Journal (JIIS)	Systematic review of cold-start methods	Preference elicitation and hybrids are most common	Few comparisons on small catalogues	Genre choice at sign-up

#	Source	Year	Type	Approach / data	Key findings	Limitations	Use in AniAtlas
6	Klimashevskaia et al.	2024	Journal (UMUAI)	Survey of popularity bias	Collaborative recommenders over-promote popular items; metrics for bias	Mostly offline studies	Long-tail metric, popularity baseline
7	Zangerle, Bauer	2022	Journal (ACM CSUR)	Recommender evaluation framework	Offline + beyond-accuracy + user studies; baselines	General, not domain-specific	Structure of the verification plan
8	Choi, Kang, Lee, Kim	2023	Conference (CHI)	Interviews with 14 YouTube creators + design workshop	Algorithm is opaque; creators feel stress and unfairness	Small qualitative sample	Creator-side motivation
9	Rieder et al.	2023	Journal (Social Media + Society)	153k channels, 137M videos	Strong inequality; small creators rely on crowdfunding	Only channels with ≥100k subscribers	Evidence of unequal visibility
10	Bishop	2019	Journal (New Media & Society)	Ethnography of YouTube creators	“Algorithmic gossip” shapes creator behaviour	Beauty vlogger community	Creator-side motivation
11	Christian	2018	Monograph (NYU Press)	History and economics of web TV	Open distribution vs. visibility burden	Pre-2018 data	Framing of independent web series
12	Cunningham, Craig	2019	Monograph (NYU Press)	Social media entertainment industry	Creators depend on platform infrastructure	Broad, not animation-specific	Context of the problem
13	Ricci, Rokach, Shapira (eds.)	2022	Handbook (Springer)	Comprehensive recommender reference	Taxonomy of methods and evaluation	Textbook level	Method selection
14	Cakmak, Agarwal, Poudel	2025	Preprint (arXiv)	685k YouTube Shorts videos analysed	Popularity bias in recommendations	Shorts only, preprint	Supports popularity baseline choice

Scientific sources: 14 (journal articles, conference papers, monographs, handbook). Of these, 7 were published in 2021–2025 and 1 in 2020; the other 6 (2011–2019) are foundational works that define the methods used. Supporting documentation and data sources: S1–S7.

3.4 Limitations the project takes into account

1. Interaction data will be sparse, so collaborative filtering is postponed [5, 6].
2. Content-based recommendations risk over-specialisation, so diversity is measured [1, 7].
3. Metadata comes from a quota-limited external API, so caching and scheduled updates are required [S5].
4. Video availability is controlled by YouTube, so the system must detect removed videos [S6].

References

Scientific

1. Lops, P., de Gemmis, M., Semeraro, G. (2011). Content-based recommender systems: State of the art and trends. In *Recommender Systems Handbook*, Springer, 73–105.

https://doi.org/10.1007/978-0-387-85820-3_3
2. Covington, P., Adams, J., Sargin, E. (2016). Deep neural networks for YouTube recommendations. *Proc. 10th ACM RecSys*, 191–198. https://doi.org/10.1145/2959100.2959190
3. Gomez-Uribe, C. A., Hunt, N. (2015). The Netflix recommender system: Algorithms, business value, and innovation. *ACM TMIS*, 6(4), 13. https://doi.org/10.1145/2843948
4. Deldjoo, Y., Schedl, M., Cremonesi, P., Pasi, G. (2020). Recommender systems leveraging multimedia content. *ACM Computing Surveys*, 53(5), 1–38. https://doi.org/10.1145/3407190
5. Panda, D. K., Ray, S. (2022). Approaches and algorithms to mitigate cold start problems in recommender systems: a systematic literature review. *Journal of Intelligent Information Systems*, 59(2), 341–366. https://doi.org/10.1007/s10844-022-00698-5
6. Klimashevskaia, A., Jannach, D., Elahi, M., Trattner, C. (2024). A survey on popularity bias in recommender systems. *User Modeling and User-Adapted Interaction*. https://arxiv.org/abs/2308.01118
7. Zangerle, E., Bauer, C. (2022). Evaluating recommender systems: Survey and framework. *ACM Computing Surveys*, 55(8), 170. https://doi.org/10.1145/3556536
8. Choi, Y., Kang, E. J., Lee, M. K., Kim, J. (2023). Creator-friendly algorithms: Behaviors, challenges, and design opportunities in algorithmic platforms. *Proc. CHI '23*. https://doi.org/10.1145/3544548.3581386
9. Rieder, B., Borra, E., Coromina, Ò., Matamoros-Fernández, A. (2023). Making a living in the creator economy: A large-scale study of linking on YouTube. *Social Media + Society*, 9(2). https://doi.org/10.1177/20563051231180628
10. Bishop, S. (2019). Managing visibility on YouTube through algorithmic gossip. *New Media & Society*, 21(11–12), 2589–2606. https://doi.org/10.1177/1461444819854731
11. Christian, A. J. (2018). *Open TV: Innovation beyond Hollywood and the Rise of Web Television*. NYU Press.
12. Cunningham, S., Craig, D. (2019). *Social Media Entertainment: The New Intersection of Hollywood and Silicon Valley*. NYU Press.
13. Ricci, F., Rokach, L., Shapira, B. (Eds.) (2022). *Recommender Systems Handbook* (3rd ed.). Springer.
14. Cakmak, M. C., Agarwal, N., Poudel, D. (2025). Investigating algorithmic bias in YouTube Shorts. arXiv:2507.04605. https://arxiv.org/abs/2507.04605

Supporting sources (documentation, data, industry)

- S1. YouTube Culture & Trends (2024, Oct 28). Set in motion: The new wave of indie animation. https://blog.youtube/culture-and-trends/indie-animation/
- S2. YouTube Official Blog (2024, Nov 21). The Amazing Digital Circus. https://blog.youtube/creator-and-artist-stories/amazing-digital-circus/
- S3. Wikipedia. Hazbin Hotel. https://en.wikipedia.org/wiki/Hazbin_Hotel
- S4. Amazon MGM Studios (2023). Prime Video announces premiere date and guest cast for Hazbin Hotel. https://press.amazonmgmstudios.com/us/en/press-release/prime-video-announces-premiere-date-and-guest-cast
- S5. Google for Developers. YouTube Data API overview (quota). https://developers.google.com/youtube/v3/getting-started
- S6. Google for Developers. YouTube IFrame Player API. https://developers.google.com/youtube/iframe_api_reference
- S7. MyAnimeList forum: database guidelines discussion. https://myanimelist.net/forum/message/58190915?goto=topic

4. Existing System Comparison

4.1 Selected systems

Four systems were selected. Together they cover the ways viewers currently find and track independent animated web series:

System	Why it is relevant
YouTube	Where most independent animation is actually published and watched
IMDb	The largest general film and TV database with ratings and watchlists; it lists some web series
MyAnimeList (MAL)	The largest specialised animation catalogue with ratings, lists and recommendations
Fandom wikis	Fan-made wikis that often hold the most detailed information about a single independent series

No direct competitor was found, that is, no platform specialised in cataloguing independent animated web series from open platforms. This is why the comparison includes neighbouring categories.

4.2 Comparison

Legend: ✅ supported · ◐ partly · ❌ not supported

#	Criterion	YouTube	IMDb	MyAnimeList	Fandom wikis	AniAtlas (planned)
C1	Focus on independent web animation	◐ hosts it, but mixed with all content	❌ general films/TV	❌ only professionally produced Asian animation [S7]	◐ one wiki per series	✅
C2	Structured metadata (genre, format, status, episodes)	❌ only title, description, playlists	✅	✅	◐ free-form, inconsistent between wikis	✅
C3	Search and filtering by genre and tags	❌ keyword search only	◐ advanced search, but not specific to web animation	✅	❌	✅
C4	Watching inside the platform	✅	❌ links out	❌	❌	✅ via official embed
C5	Ratings and reviews of a whole series	❌ likes and comments per video	✅	✅	❌	✅
C6	Personal lists and progress tracking	◐ playlists, “Watch later”	✅ watchlist	✅ detailed lists with episode progress	❌	✅
C7	Side-by-side comparison of titles	❌	❌	❌	❌	✅

#	Criterion	YouTube	IMDb	MyAnimeList	Fandom wikis	AniAtlas (planned)
C8	Recommendations transparent to the user (based on stated genres)	❌ opaque, engagement-based [2, 8]	◐ “More like this”	◐ user-made recommendations	❌	✅
C9	Cross-series discovery across many authors	◐ algorithmic, favours popular content [6, 14]	◐	❌ for this content	❌	✅
C10	Free for viewers	✅	✅	✅	✅	✅

4.3 Evidence

Claims in the table must be supported by evidence. Before submission, the team checks each cell on the live service and saves screenshots to `docs/evidence/existing-systems/` with the check date in the file name, for example `imdb-search-2026-10-23.png`. Already supported by documents:

- MAL scope (C1): database guidelines limit entries to professionally produced animation made in Japan, with China and Korea also accepted [S7].
- YouTube recommendations (C8, C9): engagement-driven deep recommender [2]; popularity bias [6, 14].

Example of an independent series to use in the checks: *The Amazing Digital Circus* (GLITCH). Check whether it is listed, filterable and rated in each system.

4.4 Conclusion

Each system covers part of the need, but none combines (1) a focus on independent web animation, (2) structured metadata with filtering, and (3) legal in-platform watching. AniAtlas fills this gap by combining the catalogue features of IMDb and MAL with YouTube’s embed playback. Its distinctive features are side-by-side comparison (C7) and transparent genre-based recommendations (C8).

5. Technology Comparison

Each decision is evaluated against requirements from Section 6 and the team's skills. Scores are on a 1–5 scale; the selected option is in **bold**.

5.1 Server platform

Criterion (weight)	Java 21 + Spring Boot 3	Go + Gin	Node.js + NestJS
Team experience (×3)	5 — both members studied Java and design patterns	3 — some Go experience	2
Ecosystem for REST, security, ORM, scheduling (×3)	5 — Spring Web, Security, Data JPA, `@Scheduled`	3 — separate libraries for each	4
Support for modular architecture (×2)	5	4	4
Performance (×1)	4	5	3
Weighted total	**44**	31	29

Decision: Spring Boot. Authentication and role-based access (NFR-2), scheduled YouTube synchronisation (FR-14) and the relational data model (FR-1, FR-2) are all covered by mature Spring modules, and the team already knows Java.

5.2 Data storage

Criterion (weight)	PostgreSQL 16	MySQL 8	MongoDB 7
Fit for a relational model with many-to-many links (×3)	5	5	2 — embedded documents complicate joins
Built-in full-text search (×2)	5 — `tsvector`, GIN indexes	3	4
Array/JSON support for genre vectors and API payloads (×2)	5 — arrays, JSONB	3	5
Team experience (×1)	4	4	4 — Advanced Databases (NoSQL) course
Weighted total	**39**	31	28

Decision: PostgreSQL. Catalogue search (FR-4, NFR-1) runs on built-in full-text indexes without an extra search engine.

Cache: Spring Cache (Caffeine) is used in the MVP. Redis is considered if several backend instances are deployed.

5.3 Component interaction

Criterion (weight)	REST (JSON over HTTP)	GraphQL	gRPC
Browser support without proxies (×3)	5	5	2 — needs gRPC-Web
Simplicity and tooling (OpenAPI, Swagger UI) (×2)	5	3	3
Flexible queries for the comparison page (×1)	3	5	3

Criterion (weight)	REST (JSON over HTTP)	GraphQL	gRPC
Team experience (×2)	5	2	2
Weighted total	**38**	30	19

Decision: REST with an OpenAPI specification. The backend modules communicate in-process in the MVP. If they are split into separate services later, REST stays the protocol.

5.4 Frontend

Criterion (weight)	Vanilla HTML/CSS/JS	React + Vite	Thymeleaf (server-side)
Interactive filtering without page reloads (×3)	3	5	2
Reusable components: cards, player, filters (×2)	2	5	3
Team experience (×2)	5	3	3
Separation from the backend (×1)	5	5	1
Weighted total	28	**36**	19

Decision: the whole frontend is built with React and Vite. A single-page application gives instant filtering without page reloads (FR-5), and reusable components (title card, filter panel, YouTube player wrapper, rating widget) keep the five main screens consistent.

Concern	Library
Build and dev server	Vite
Routing (catalogue, title, compare, profile, submit, moderation)	React Router
Server state and caching of API calls	TanStack Query
YouTube playback	Wrapper component over the IFrame Player API
Forms and validation (sign-up, submission)	React Hook Form
Unit and component tests	Vitest + React Testing Library

5.5 Recommendation method

Criterion	Content-based (genre/tag vectors)	Collaborative filtering	Hybrid / neural
Works with few users (cold start) [5]	✅	❌	◐
Treats new and unpopular titles fairly [6]	✅	❌ popularity bias	◐
Explainable to the user (“because you like Horror”)	✅	◐	❌
Implementation effort	Low	Medium	High

Decision: content-based filtering in the MVP. Collaborative signals are added as a hybrid if enough ratings are collected (future work).

5.6 Other choices

Area	Choice	Alternatives considered	Reason
Video playback	YouTube IFrame Player API	Downloading or re-hosting video	Legal compliance (NFR-6); views stay with the author
Metadata source	YouTube Data API v3	Web scraping	Officially supported, within ToS, quota-managed
Migrations	Flyway	Liquibase, Hibernate `ddl-auto`	Versioned SQL scripts, simple
Authentication	JWT (stateless) with Spring Security	Server-side sessions; login via Google OAuth	Works with a separate React SPA and stateless backend containers; Google login can be added later without changing the API (NFR-2)
Deployment	Docker Compose	Kubernetes, bare metal	Sufficient for an MVP; one-command start (NFR-4)
CI	GitHub Actions	GitLab CI, Jenkins	Repository is on GitHub

6. Requirements Specification

Priorities use the MoSCoW scheme:

- **M** — Must: required for the MVP.
- **S** — Should: important.
- **C** — Could: if time allows.
- **W** — Won't in this version.

Every *Must* requirement has an acceptance criterion that can be checked by a test or a demonstration. The "Source" column links each requirement to evidence (Section 2) or literature (Section 3).

6.1 Actors

- **Guest**: unauthenticated visitor.
- **Viewer**: registered user.
- **Author**: creator who submits titles.
- **Moderator**: approves content and manages reference data.
- **YouTube**: external system that provides the Data API and the embed player.

6.2 Functional requirements

ID	Requirement	Priority	Acceptance criterion	Source
FR-1	The system stores titles with name, description, poster, format (series/film/short), status (ongoing/completed/hiatus), release year, authors, genres and tags	M	A title created with all fields via API is returned unchanged by `GET /titles/{id}`	D5, A2
FR-2	A series consists of seasons and episodes; each episode references one YouTube video ID	M	A title with 2 seasons × 3 episodes is returned with correct order and video IDs	D1
FR-3	A title page shows metadata, the episode list, average rating and reviews	M	UI test: all elements are visible for a seeded title	—
FR-4	Full-text search by title and author name	M	Searching "digital circus" returns *The Amazing Digital Circus* first in the seeded catalogue	A1
FR-5	Filtering by genres, tags, format, status and release year, combinable	M	Each filter and a combination of 3 filters return only matching titles (integration test on seed data)	A2
FR-6	Sorting by rating, date added and release year	S	The order of results matches the chosen key	—
FR-7	Episodes are played through the official YouTube IFrame embed player	M	An episode plays on the title page; no video file is requested from AniAtlas servers (checked in browser network tab)	A3, NFR-6

ID	Requirement	Priority	Acceptance criterion	Source
FR-8	Episodes whose video is no longer available are marked as unavailable	S	After a video ID is made invalid, the sync job sets `available = false` and the UI shows a label	S6
FR-9	Registration and login with email and password	M	A user can register, log in and receive a JWT; invalid credentials return 401	—
FR-10	A viewer can rate a title (1–10) and write a review	M	The rating is saved; the average rating updates; a second rating by the same user replaces the first	A4
FR-11	A viewer keeps personal lists: watching, planned, completed, dropped, with last watched episode	M	Moving a title between lists is reflected in `GET /me/lists`	A4
FR-12	A viewer can compare 2–3 titles side by side	S	The comparison page shows genres, format, episode count, total duration and rating for each selected title	C7
FR-13	An author or moderator submits a title by YouTube playlist URL; metadata and episodes are imported automatically	M	Submitting a valid public playlist creates a PENDING title with all its videos as episodes; an invalid URL returns 400	A6, S5
FR-14	A scheduled job refreshes metadata of published titles within the API quota	S	The job runs daily at 03:00 (cron set by `YOUTUBE_SYNC_CRON`), adds new episodes, marks removed videos and logs quota units used; daily usage stays below `YOUTUBE_DAILY_QUOTA_LIMIT` (9,000 units, a buffer under the 10,000 quota)	S5
FR-15	Submitted titles appear publicly only after moderator approval	M	A PENDING title is not returned to guests; after approval it is	—
FR-16	The system builds a genre/tag preference profile from the user’s ratings, lists and chosen genres	M	Unit test: profile vector matches the expected values for a fixed set of ratings	[1, 5]
FR-17	The system recommends unseen titles ranked by similarity to the profile, with an explanation (“because you like …”)	M	Recommendations exclude titles already in the user’s lists; each item has an explanation; offline precision@10 is above the popularity baseline (Section 9)	[1, 6, 7]
FR-18	New users choose at least 3 favourite genres at sign-up (cold start)	M	A new user with no ratings receives non-empty recommendations matching the chosen genres	[5]
FR-19	The moderator manages genres and tags and can edit or reject titles	S	CRUD for genres/tags is available only to the MODERATOR role	—

ID	Requirement	Priority	Acceptance criterion	Source
FR-20	Notify the viewer about new episodes of titles in “watching”	S	When the sync job finds a new episode, a notification is created for every user who has the title in “watching”	Interviews F3 (R3)
FR-21	Sources other than YouTube (Vimeo, Bilibili)	W	—	—

6.3 Non-functional requirements

ID	Category	Requirement	Priority	Acceptance criterion
NFR-1	Performance	Catalogue search and filtering respond fast	M	p95 latency < 500 ms with 10,000 titles and 20 concurrent users (k6 load test)
NFR-2	Security	Passwords are hashed; access is role-based	M	Passwords stored with BCrypt; endpoints for other roles return 403; OWASP ZAP baseline scan has no high-risk findings
NFR-3	External API	YouTube API usage stays within the free quota	M	Quota counter in logs; < 10,000 units/day during a 7-day test run
NFR-4	Portability	The system starts with one command	M	`docker compose up` on a clean machine starts all services; README instructions are enough
NFR-5	Usability	Responsive interface	S	Key pages are usable at 375 px and 1440 px width; SUS score ≥ 70 in the user test
NFR-6	Legal	No video is downloaded or re-hosted	M	Code review and network check: only YouTube embed URLs are used
NFR-7	Maintainability	Automated tests	S	Backend line coverage ≥ 60%; CI runs tests on every pull request
NFR-8	Privacy	Minimal personal data	M	Only email, username and password hash are stored; no secrets in the repository

6.4 Traceability

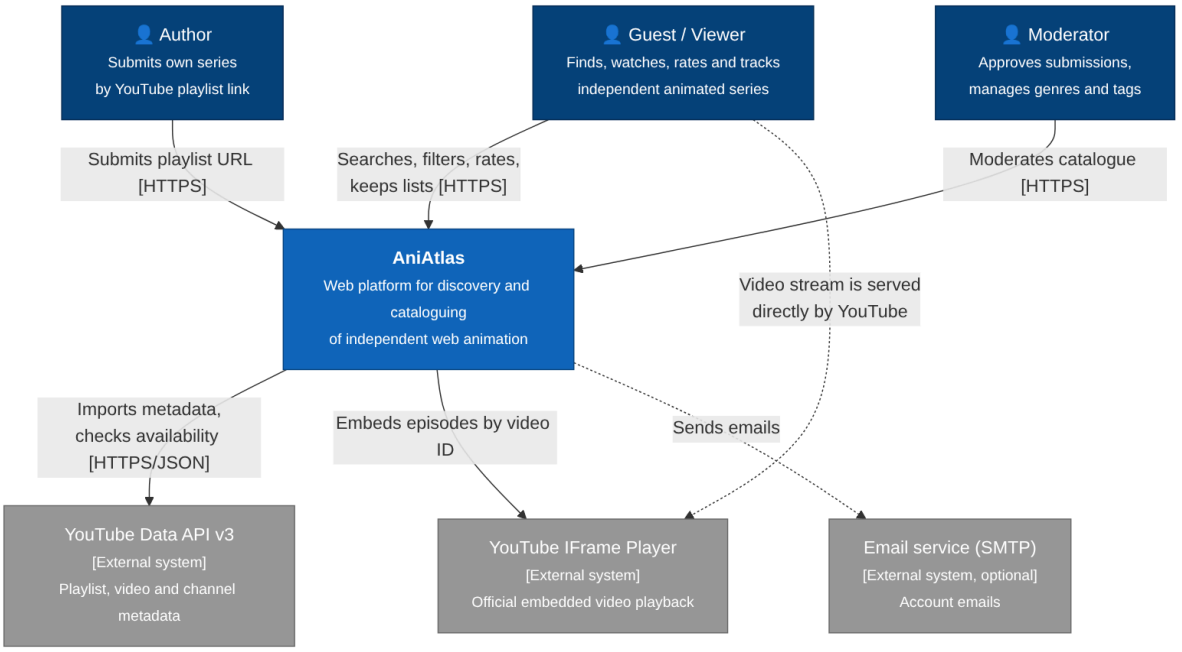
Requirement group	Use cases (Section 7)	Components (Section 7)	Tests (Section 9)
FR-1…FR-6	UC-1 Search catalogue, UC-2 View title	Catalogue module	T-CAT
FR-7, FR-8	UC-3 Watch episode	Frontend player, Aggregation module	T-PLAY
FR-9…FR-12	UC-4…UC-7	Users module	T-USR
FR-13…FR-15, FR-19	UC-8 Submit title, UC-9 Moderate	Aggregation, Catalogue	T-AGG
FR-16…FR-18	UC-10 Get recommendations	Recommendation module	T-REC, offline evaluation

7. Architecture and Diagrams

All diagrams are stored as editable source files in `diagrams/src`: Mermaid `.mmd` and Graphviz `.dot`. Readable exports are in `diagrams/export` as PNG and SVG. To regenerate them:

```
        npx -p @mermaid-js/mermaid-cli mmdc -i diagrams/src/system-context.mmd -o
diagrams/export/system-context.png
        dot -Tpng diagrams/src/use-case.dot -o diagrams/export/use-case.png
```

7.1 System context diagram

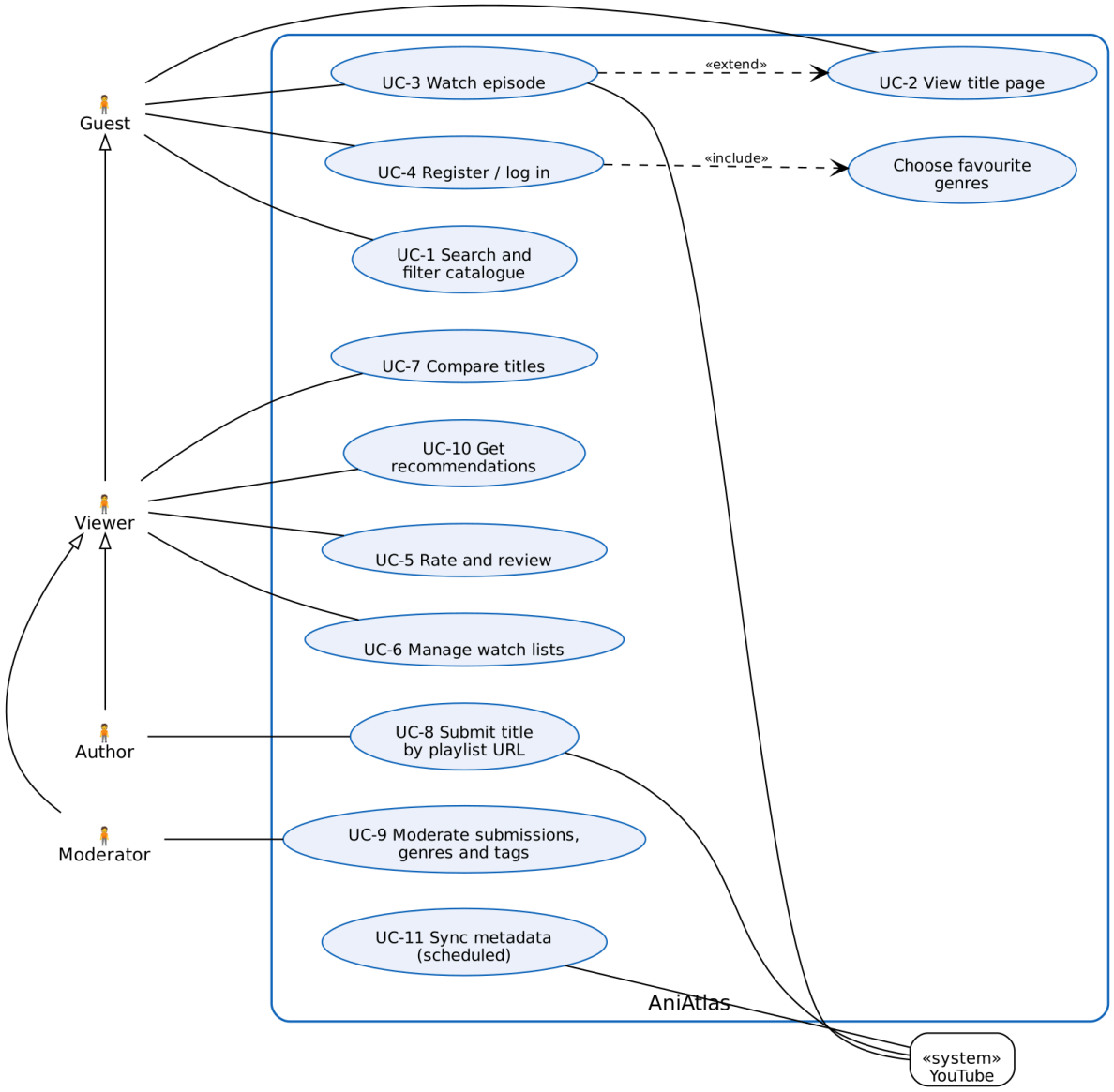


System context

AniAtlas is used by three kinds of people: viewers (guests and registered users), authors and moderators. It depends on two external systems from YouTube:

- The **Data API v3** is used server-side to import playlist and video metadata and to check video availability.
- The **IFrame Player** is used client-side for playback. The video stream goes directly from YouTube to the viewer's browser, so AniAtlas never stores or proxies video (NFR-6).

7.2 Use case diagram



Use cases

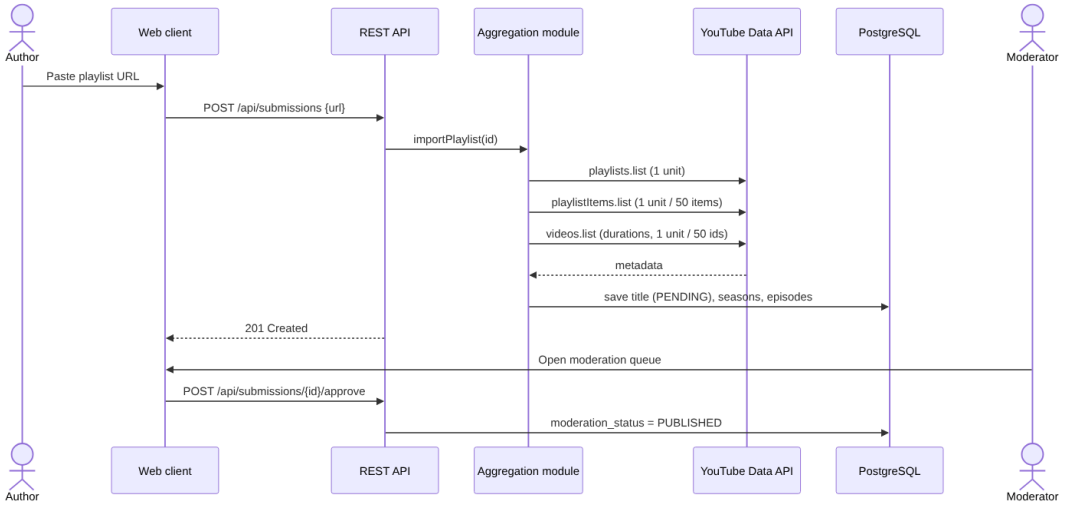
Use case	Actor	Requirements
UC-1 Search and filter catalogue	Guest	FR-4, FR-5, FR-6
UC-2 View title page	Guest	FR-3
UC-3 Watch episode	Guest	FR-7, FR-8
UC-4 Register / log in (includes choosing favourite genres)	Guest	FR-9, FR-18
UC-5 Rate and review	Viewer	FR-10
UC-6 Manage watch lists	Viewer	FR-11
UC-7 Compare titles	Viewer	FR-12
UC-8 Submit title by playlist URL	Author	FR-13
UC-9 Moderate submissions, genres and tags	Moderator	FR-15, FR-19
UC-10 Get recommendations	Viewer	FR-16, FR-17
UC-11 Sync metadata (scheduled)	YouTube (system)	FR-8, FR-14

Actor generalisation: Viewer inherits Guest; Author and Moderator inherit Viewer.

Main scenario: UC-8 Submit title

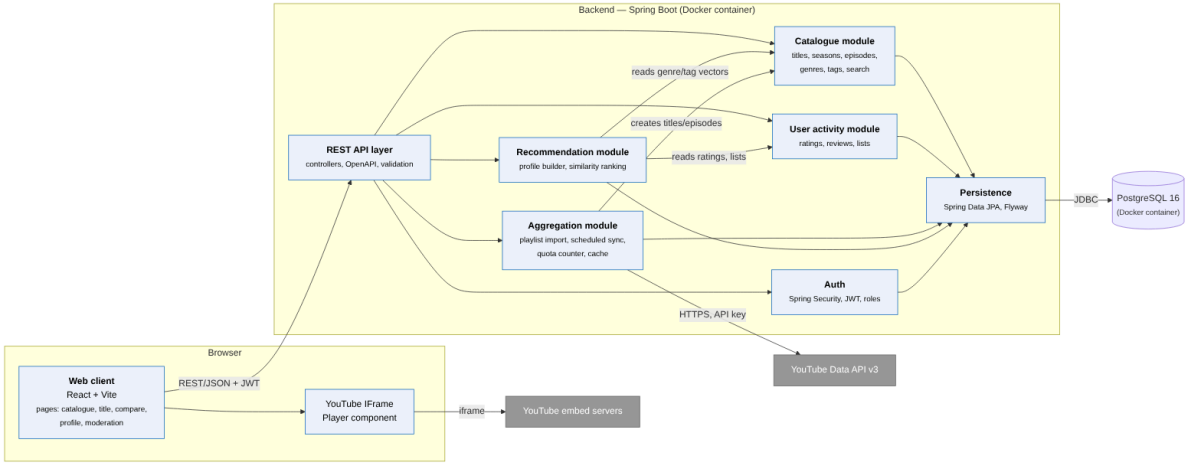
1. The author pastes a YouTube playlist URL and chooses genres and tags.
2. The system validates the URL and fetches the playlist, its items and video details from the Data API.
3. The system creates the title with status PENDING, one season, and an episode for each video.
4. The moderator reviews the title in the queue, edits it if needed, and approves it.
5. The title becomes visible in the catalogue.

Alternative flows: if the URL is invalid or the playlist is private, the system shows an error (400). If the daily quota is exhausted, the submission is queued for the next day.



Sequence

7.3 Component diagram (draft)



Components

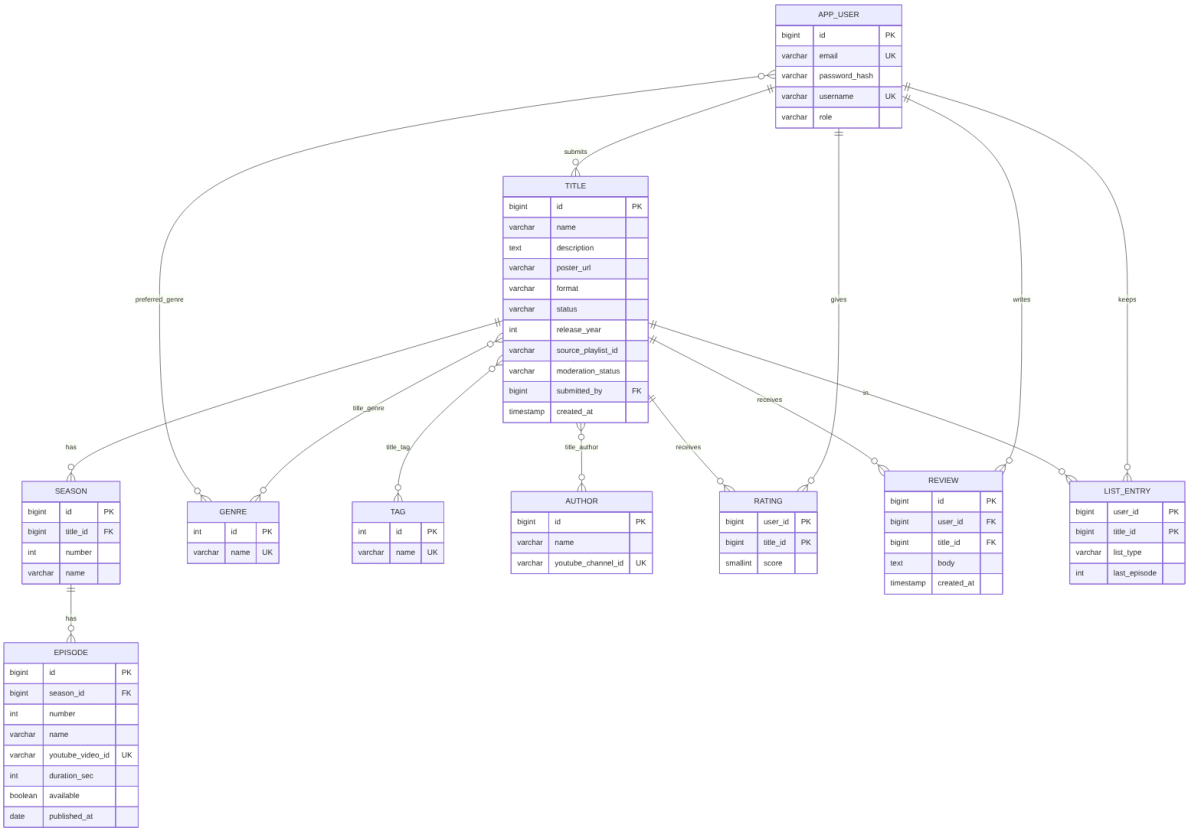
Component	Responsibility	Main requirements
Web client (React)	Pages: catalogue, title, comparison, profile, moderation	FR-3, FR-5, FR-12
YouTube Player component	Wraps the IFrame Player API; shows “unavailable” state	FR-7, FR-8
REST API layer	Controllers, validation, OpenAPI docs, error handling	All

Component	Responsibility	Main requirements
Auth	Registration, login, JWT, roles USER/AUTHOR/MODERATOR	FR-9, NFR-2
Catalogue module	Titles, seasons, episodes, genres, tags; full-text search and filters	FR-1…FR-6, FR-19
User activity module	Ratings, reviews, lists	FR-10, FR-11
Aggregation module	Playlist import, scheduled sync, quota counter, response cache	FR-13, FR-14, NFR-3
Recommendation module	Builds preference profiles, ranks titles, explains results	FR-16…FR-18
Persistence	JPA repositories, Flyway migrations	—
PostgreSQL	Data storage, full-text index	NFR-1

Architectural style: a modular monolith. Each module is a separate Java package with its own service interfaces, and modules communicate only through those interfaces. This keeps deployment simple for the MVP while still allowing modules to be split into separate services later. Splitting the Aggregation module first is the most likely step, because of its scheduled load.

Deployment: Docker Compose with three containers: `frontend` (nginx serving the built React app), `backend` (Spring Boot) and `db` (PostgreSQL). Secrets (DB password, YouTube API key, JWT secret) are passed as environment variables from `.env`, which is not committed (see `.env.example`).

7.4 Data model



ER model

- `RATING` and `LIST_ENTRY` use the composite key `(user_id, title_id)`, so each user has at most one rating and one list entry per title.
- `EPISODE.youtube_video_id` is unique.

- `TITLE` has a full-text index over `name` and `description` (`tsvector`, GIN).

7.5 Recommendation algorithm

1. Each title is a binary vector over genres and tags, **g**.
2. User profile **p** = Σ (rating − user's mean rating) · **g** over rated titles + 1.0 · **g** for titles in the "watching" or "completed" lists + 1.0 for each genre chosen at sign-up.
3. Score(title) = cos(**p**, **g**) + 0.05 · normalised average rating; titles already in the user's lists are excluded.
4. The explanation lists the top 2 shared genres or tags.

7.6 Alignment check

Use case	Components involved	Data entities
UC-1	Web client → API → Catalogue	TITLE, GENRE, TAG
UC-3	Web client → Player → YouTube	EPISODE
UC-5/6	Web client → API → Auth → User activity	RATING, REVIEW, LIST_ENTRY
UC-8	Web client → API → Aggregation → YouTube Data API	TITLE, SEASON, EPISODE, AUTHOR
UC-10	Web client → API → Recommendation → Catalogue, User activity	APP_USER, RATING, LIST_ENTRY, GENRE

The draft REST API is described in api.md.

8. UX / Prototype

Interactive prototype (React): `prototype/`. It is built with React 18, React Router and Vite, the same stack and component structure planned for `frontend/` (see Section 5.4). Run it with Node.js 18+: `cd prototype && npm install && npm run dev`, then open http://localhost:5173.

Visual language. The interface borrows from the animator's desk: light-table paper, non-photo-blue construction lines and red pencil accents. Title cards are drawn as cels with registration peg holes, placeholder artwork is an onion-skin pencil test generated per title (no copyrighted posters), and the episode list is laid out like an exposure sheet. Light and dark themes follow the system setting.

The prototype uses fictional sample titles. The YouTube player is shown as a placeholder. In the real system this area is the official YouTube IFrame player.

8.1 Key user scenarios covered

Screen	Scenario	Use case	Requirements	What to try
Catalogue	Find a title by genre, format and status; search by name or author; sort	UC-1	FR-4, FR-5, FR-6	Select *Horror* + *Comedy*, then *Series*
Title page	View metadata, switch episodes, see an unavailable episode, rate 1–10, add to a list, read reviews	UC-2, UC-3, UC-5, UC-6	FR-3, FR-7, FR-8, FR-10, FR-11	Open *The Quiet Floor* → Tape 3
Compare	Compare 2–3 titles side by side; highest values highlighted; the Compare checkbox is disabled once 3 titles are selected	UC-7	FR-12	Tick Compare on titles in the catalogue
My lists	Recommendations with "because you like …" explanations; lists with episode progress	UC-6, UC-10	FR-11, FR-16…FR-18	Switch between list tabs
Submit a series	Paste a playlist URL, see the imported preview and quota usage, send to the moderator	UC-8	FR-13, FR-15	Clear the URL and press Import to see the validation error

8.2 Component structure

Component	Used in	Becomes in `frontend/`
`TitleCard`, `Poster`	Catalogue, compare, lists	Same card; `Poster` replaced by the YouTube thumbnail from `GET /api/titles`
`FilterGroup` (checkbox filters with counts)	Catalogue	Same, filter state in URL query parameters

Component	Used in	Becomes in `frontend/`
`YouTubePlayer`	Title page	Wrapper over the YouTube IFrame Player API
`RatingWidget`	Title page	Same, calls `PUT /api/titles/{id}/rating`
`recommend()`	My lists	Replaced by `GET /api/me/recommendations`

8.3 Interaction with the external system (YouTube)

Direction	Call	Input	Output
Backend → Data API	`playlists.list?id=<playlistId>&part=snippet`	Playlist ID from the submitted URL	Title, description, thumbnails, channel
Backend → Data API	`playlistItems.list?playlistId=…&maxResults=50`	Playlist ID	Video IDs in order → episodes
Backend → Data API	`videos.list?id=<up to 50 ids>&part=contentDetails,status`	Video IDs	Duration, `embeddable`, availability
Browser → IFrame Player	`https://www.youtube.com/embed/<videoId>`	Video ID	Playback served by YouTube

The REST API between the web client and the backend, with request and response examples, is in api.md.

8.4 Next steps

- Show the prototype to interview respondents (Section 2, question 10) and record their feedback.
- Build high-fidelity mockups in Figma for the screens that change after the feedback.

9. Verification and Evaluation Plan

The project aim (Section 1.3) is demonstrated if: (a) every *Must* requirement passes its acceptance criterion, (b) the recommender is evaluated against the baselines in 9.4 and the results are reported whatever they show, and (c) users can complete the key scenarios.

Status (24 Oct 2026). No application code exists yet, so **all automated tests below are planned, none executed**. Executed so far: a manual walkthrough of the React prototype (all five screens, 1360 px and 390 px width, light and dark theme). Results of executed tests will be stored in `tests/results/` and listed in a separate "Executed" table from pre-defense 2 onwards.

9.1 Testing plan

Level	What	Tools	When	Pass condition	Status
Unit	Services: search query builder, profile builder, similarity ranking, playlist URL parser	JUnit 5, Mockito	Every PR (CI)	All pass; backend line coverage ≥ 60% (JaCoCo)	Planned
Component	React components: filters, title card, rating widget, player wrapper	Vitest, React Testing Library	Every PR (CI)	All pass	Planned
Integration	REST endpoints with a real database	Spring Boot Test, Testcontainers (PostgreSQL)	Every PR (CI)	All acceptance tests for Must requirements pass (see traceability in 9.6)	Planned
External API	YouTube client against recorded responses	WireMock	Every PR	Import works on recorded playlists; quota counter correct	Planned
End-to-end	Key scenarios in the browser	Playwright	Before each pre-defense	UC-1, UC-3, UC-5, UC-6, UC-8, UC-10 pass	Planned
Performance	Search and filter under load	k6	Stage 6	p95 < 500 ms, 10,000 titles, 20 virtual users (NFR-1)	Planned
Security	Role checks, dependency and baseline scan	Integration tests, OWASP ZAP baseline	Stage 6	No high-risk findings (NFR-2)	Planned
Usability	Moderated user test	SUS questionnaire, task success	Stage 6	SUS ≥ 70; ≥ 80% task success (NFR-5)	Planned

Test IDs map to requirement groups: T-CAT (FR-1…6), T-PLAY (FR-7, 8), T-USR (FR-9…12), T-AGG (FR-13…15, 19), T-REC (FR-16…18).

9.2 Outcome metrics

Aim component	Metric	Target
Catalogue completeness	Number of real independent titles imported	≥ 50 titles, ≥ 300 episodes
Search quality	Precision@5 for 20 predefined queries (relevance judged by both team members)	≥ 0.8
Recommendation accuracy	Precision@10, nDCG@10 (offline) [7]	Reported against the popularity and random baselines
Recommendation fairness	Catalogue coverage; share of long-tail titles in top-10 [6]	Coverage ≥ 40%; long-tail share higher than baseline
Performance	p95 latency of `/api/titles`	< 500 ms
API efficiency	YouTube quota units per day	< 10,000
Usability	SUS score; task completion rate	≥ 70; ≥ 80%

9.3 Measurement conditions

- **Environment:** Docker Compose on one machine (record CPU and RAM in the report). PostgreSQL is seeded with real titles plus synthetic titles up to 10,000 for load tests.
- **Offline recommendation evaluation:** at least 20 participants (classmates and interview respondents) each rate at least 15 titles. For each user, 20% of their positive ratings (score ≥ 7) are hidden. The model must rank the hidden titles among the top 10. The evaluation is repeated over 5 random splits and the mean ± standard deviation is reported.
- **User test:** 5–8 participants who did not take part in development, 6 tasks each (find a horror short, watch an episode, rate a title, add it to a list, compare two titles, open recommendations). After the tasks they fill in the SUS questionnaire.
- All raw results are stored in `tests/results/`.

9.4 Baselines for comparison

Component	Baseline	Why
Recommender	**Popularity**: top titles by number of ratings and views, excluding seen titles	The standard simple baseline [7]; it reflects how general platforms surface content [6, 14]
Recommender	**Random** among unseen titles	Lower bound
Discovery task	Finding the same titles via **YouTube search**	Shows the benefit of structured filters (task time and success compared in the user test)

9.5 Evidence produced

The evidence consists of test reports from CI, a JaCoCo coverage report, k6 output, an offline evaluation notebook (`tests/recsys-eval/`), and SUS results. Together they are the basis for the “Results” chapter of the thesis.

9.6 Traceability: Must requirements → tests → evidence

Requirement	Acceptance test (planned)	Evidence to be produced	Status
FR-1 Store titles	T-CAT-01: create title via API, read it back	CI test report	Planned

Requirement	Acceptance test (planned)	Evidence to be produced	Status
FR-2 Seasons and episodes	T-CAT-02: 2 seasons × 3 episodes returned in order	CI test report	Planned
FR-3 Title page	E2E-UC2: all page elements visible	Playwright report	Planned (prototype screen exists)
FR-4 Full-text search	T-CAT-03: known title ranked first	CI test report	Planned
FR-5 Combined filters	T-CAT-04: 3 filters return only matching titles	CI test report	Planned (prototype screen exists)
FR-7 Embed playback	E2E-UC3: episode plays; no video requests to backend	Playwright report + network log	Planned
FR-9 Registration and login	T-USR-01: register, login, 401 on bad password	CI test report	Planned
FR-10 Rating and review	T-USR-02: rating saved, average updated, re-rating replaces	CI test report	Planned
FR-11 Watch lists	T-USR-03: move title between lists	CI test report	Planned
FR-13 Playlist submission	T-AGG-01: valid playlist → PENDING title; invalid → 400 (WireMock)	CI test report	Planned
FR-15 Moderation	T-AGG-02: PENDING hidden from guests, visible after approval	CI test report	Planned
FR-16 Preference profile	T-REC-01: profile vector for fixed ratings	CI test report	Planned
FR-17 Recommendations	T-REC-02: seen titles excluded, explanation present; offline evaluation (9.3)	CI report + evaluation notebook	Planned
FR-18 Cold start	T-REC-03: new user with 3 genres gets recommendations	CI test report	Planned
NFR-1 Performance	k6 load test	k6 output	Planned
NFR-2 Security	Role tests (403), ZAP baseline	CI report, ZAP report	Planned
NFR-3 API quota	Quota counter in logs over 7 days	Log export	Planned
NFR-4 Portability	`docker compose up` on a clean machine	Screen recording	Planned
NFR-6 Legal	Code review: only embed URLs	PR review	Planned
NFR-8 Privacy	No secrets in repo; only email, username, password hash stored	`.gitignore`, schema review	Partly done (`.gitignore`, `.env.example`)

10. Project Management and Risks

10.1 Tools and workflow

- **Repository:** https://github.com/nuulun/AniAtlas. `main` is the reviewed baseline.
- **Task tracking:** YouTrack project `AniAtlas` (prefix ANI): https://aniatlas.youtrack.cloud/issues/ANI. Each task has an assignee, priority, due date, expected outcome and acceptance criteria.
- **Workflow:**
  1. Create a YouTrack task.
  2. Create a branch `ANI-<n>-short-name`.
  3. Make commits whose messages start with `ANI-<n>:`.
  4. Open a Pull Request that references the task and describes how the change was checked.
  5. The other team member reviews the PR.
  6. Merge, add the PR link to the task, and move the task to Done.
- **Tags:** one tag for each submitted stage (`prob-period-1`, then `pre-defense-2`, `final-pre-defense`). Tags are never moved after submission.
- **Meetings:** weekly team sync; meetings with the supervisor every two weeks.

10.2 Allocation of responsibilities

Area	Nurlan Bekov	Madiyar Armish
Analysis	Literature review, requirements	Project proposal and scope, existing system comparison
Design	System context and use case diagrams, UI prototype	Technology comparison, component diagram, ER model, API
Implementation	Frontend (React), player, search UI, recommendations UI	Backend (Spring Boot), database, YouTube integration, recommender service, Docker, CI
Quality	E2E tests, user test (SUS)	Unit/integration tests, load test, offline evaluation
Shared	Interviews (2+ each), mutual PR reviews, thesis chapters for own areas	

10.3 Milestones

#	Milestone	Main deliverables	Due
M1	Defense 1 / probationary period 1 (`prob-period-1`)	Documentation sections 1–11, interviews, React prototype, YouTrack and repository setup	24.10.2026 (done)
M2	Backend and frontend skeleton	Spring Boot + PostgreSQL + Flyway migration, React app from the prototype, Docker Compose, CI on GitHub Actions	Nov 2026
M3	Catalogue MVP	Catalogue CRUD, search, filters, title page, embed player, YouTube playlist import with mocked API tests	Dec 2026

#	Milestone	Main deliverables	Due
M4	**Pre-defense 2** (`pre-defense-2`)	M2 + M3 running and demonstrated; supervisor feedback items 1–8 closed with evidence; usability test of the prototype; additional interviews	date set by the university
M5	Recommendations	Profile, ranking, cold start, offline evaluation against baselines	after pre-defense 2
M6	Testing and evaluation	Load, security, usability study, results chapter	before final pre-defense
M7	Final pre-defense and defense (`final-pre-defense`)	Full system, thesis, presentation	date set by the university

Deliverables are assigned to the checkpoint in which they are demonstrated, not planned. Dates for M4 and M7 will be filled in when the official schedule is published.

10.4 Dependencies

Dependency	Type	Affects	Mitigation
YouTube Data API key and quota	External	FR-13, FR-14	Request a key early; cache responses; WireMock in tests
YouTube embed allowed by uploader	External	FR-7	Detect non-embeddable videos (`status.embeddable`) and link to YouTube instead
Interview respondents and test users	People	Section 2, user test	Recruit from student groups and animation communities early
Ratings data for evaluation	Data	FR-17 evaluation	Rating session with ≥ 20 participants
Backend API → Frontend	Internal	M3	Agree on the OpenAPI contract first; frontend uses mock data until endpoints are ready
Catalogue → Recommendations	Internal	M5	Seed catalogue by M3

10.5 Risks

ID	Risk	Probability	Impact	Mitigation	Owner
R1	YouTube API quota exhausted or terms change	Medium	High	Caching, batch requests (50 IDs per call), incremental sync, quota monitoring	Madiyar
R2	Videos removed or made non-embeddable	High	Medium	Availability checks in sync job; “unavailable” state	Madiyar

ID	Risk	Probability	Impact	Mitigation	Owner
R3	Too few ratings for meaningful evaluation	Medium	High	Organise rating session early; content-based method does not need many users	Both
R4	Uneven workload in a team of two	Medium	Medium	Balanced task allocation; weekly sync; YouTrack board review	Both
R5	Scope creep (extra sources, social features)	Medium	Medium	MoSCoW priorities; *Won't* items stay out	Both
R6	Copyright or ToS concerns	Low	High	Only official embeds; no downloads; legal note in README	Both
R7	Exam period reduces available time	High	Medium	Buffer before pre-defenses; prioritise *Must* requirements	Both

11. Supervisor Feedback and Feedback Log

Reviewer: Akbota Rakym · **Reviewed:** Defense 1 documentation, 23 October 2026 · **Log updated:** 24 October 2026

The supervisor reviewed the proposal, interviews, literature review, comparisons, requirements, diagrams, prototype and verification plan. Overall assessment: a coherent and promising design, with implementation and performance targets still to be demonstrated.

Following the supervisor's instruction, an item is marked **Done** only when the change can be shown in the repository. Items that require new research or running code are **Planned** for pre-defense 2 and tracked as YouTrack tasks.

#	Comment (summary)	Priority	Team decision and action	Evidence	Status
1	Sample of 3 viewer interviews is small; add viewers and, where possible, creators or moderators; keep guides, evidence and a requirements-change summary	High	**Accepted.** Interview guide, anonymised results, findings, assumption status and requirements-change summary added to Section 2. Further interviews (≥ 1 independent creator or moderator) and usability sessions scheduled	Section 2.2–2.6; `docs/interviews/`; YouTrack ANI-13	Partly done; rest Planned (pre-defense 2)
2	Define recommendation features, weighting, profile, cold start, explanation; participant recruitment; metric calculation; no claims of superiority before testing	High	**Accepted.** Claims rewritten as hypotheses (Sections 1.4, 9). Full methodology document (features, weights, profile, cold start, explanation, recruitment, metric formulas) will be written	Sections 1.4 O7, 9.1–9.2; YouTrack ANI-14	Partly done; rest Planned (pre-defense 2)

#	Comment (summary)	Priority	Team decision and action	Evidence	Status
3	Specify YouTube import: duplicates, quota, caching, retries, unavailable/non-embeddable videos, refresh, moderation, missing data; demonstrate with mocked responses	High	**Accepted.** An integration specification and WireMock fixtures for each case will be added together with the import module	YouTrack ANI-15	Planned (pre-defense 2)
4	Traceability matrix Must requirement → test → evidence; FR-19 mismatch; separate planned from executed tests	High	**Accepted.** Traceability matrix added (9.6); test table now has a Status column, all automated tests marked Planned. FR-19 exists in the full specification (Section 6) and was accidentally left out of the shortened Defense 1 document; references now match	Section 9.1, 9.6; Section 6 FR-19; YouTrack ANI-12	Done
5	Refine component diagram (module boundaries, data flows, scheduled sync, recommendations); explain JWT roles, moderator privileges, validation, API credentials	Medium	**Accepted.** Security section and refined component diagram will be updated together with the backend skeleton so that diagrams reflect the implementation	YouTrack ANI-16	Planned (pre-defense 2)

#	Comment (summary)	Priority	Team decision and action	Evidence	Status
6	Separate peer-reviewed claims from product-site claims; verify absolute competitor claims with evidence; sharpen unique value proposition	Medium	**Accepted.** Data table in Section 2.1 now labels each source type. Competitor table cells will be checked on live services with dated screenshots, absolute claims softened, and a one-sentence value proposition added	Section 2.1; YouTrack ANI-2 (screenshots), ANI-17	Partly done; rest Planned
7	Run task-based usability tests (search, comparison, watch lists, submission, embedded playback); record completion, confusing steps, changes; accessibility and mobile checks	Medium	**Accepted.** Manual walkthrough at 1360 px / 390 px, light and dark theme done. Moderated usability sessions (5 tasks, SUS) planned	Section 9 status note; YouTrack ANI-18	Partly done; rest Planned (pre-defense 2)
8	Keep tasks assigned and linked to branches, commits and PRs; at the next checkpoint demonstrate running frontend/backend skeleton, DB migration, Docker and CI	High	**Accepted.** All Defense 1 work done through YouTrack tasks ANI-1…ANI-12, branches and reviewed PRs (integration links commits to tasks). Skeleton, Flyway migration, Docker Compose and CI planned as milestone M2	GitHub PRs; YouTrack board; Section 10.3 M2; YouTrack ANI-19, ANI-20	Partly done; rest Planned (pre-defense 2)
9	Supervisor surname “Rakym”; align the “Pre-defense 1” milestone with the actual schedule and clarify deliverables per checkpoint	Required	**Accepted.** Surname corrected in README, Section 1 and the report cover. Milestones renamed (Defense 1 → pre-defense 2 → final pre-defense) with deliverables per checkpoint	README; Section 1; Section 10.3; YouTrack ANI-12	Done